Nama : NAYLA RABIA GUSTARI
NPM : 20241320034
TANGGAL : 20/10/2026
MATKUL : PEMOGRAMAN PYTHON

A. SOAL TEORI

1. *Jelaskan perbedaan mendasar antara list dan tuple dalam Python. Berikan contoh skenario di mana tuple lebih tepat digunakan daripada list, dan sebaliknya.*

- List dan tuple merupakan struktur data Python yang digunakan untuk menyimpan kumpulan data. Perbedaan mendasarnya terletak pada sifat mutabilitasnya.
- List bersifat mutable, artinya isi, urutan, dan ukurannya dapat diubah setelah list dibuat. List menggunakan tanda kurung siku [] dan cocok untuk menyimpan koleksi data yang perlu ditambah, dihapus, atau diurutkan.
- Tuple bersifat immutable, artinya data tidak dapat diubah setelah tuple dibuat. Tuple menggunakan tanda kurung biasa () dan cocok untuk menyimpan data yang bersifat tetap.

contoh kode :

```
# List

mahasiswa = ["Ani", "Budi", "Cici"]

mahasiswa.append("Dina")

print(mahasiswa)


# Tuple

koordinat = (3.14, 2.71)

print(koordinat)
```

Contoh skenario penggunaannya:

- Tuple lebih tepat digunakan untuk menyimpan koordinat geografis (latitude, longitude), tanggal (tahun, bulan, hari), atau konfigurasi tetap karena datanya tidak perlu diubah.
- List lebih tepat digunakan untuk menyimpan daftar mata kuliah yang diambil mahasiswa karena mata kuliah dapat ditambah atau dihapus selama program berjalan.

Jadi, list digunakan untuk koleksi data yang dinamis, sedangkan tuple digunakan untuk data yang konstan atau tidak boleh diubah.

2. *Mengapa set tidak mendukung pengindeksan seperti list dan tuple? Jelaskan kaitannya dengan implementasi hash table dan sifat unordered.*

- Set tidak mendukung pengindeksan karena set merupakan kumpulan data yang bersifat unordered atau tidak memiliki urutan indeks yang tetap. Set menggunakan struktur data hash table untuk menyimpan dan mencari elemen berdasarkan nilai hash, bukan berdasarkan posisi indeks seperti pada list dan tuple.
- Karena itu, elemen set tidak dapat diakses menggunakan indeks seperti set[0]. Set lebih cocok digunakan untuk menyimpan data unik, menghilangkan duplikasi, dan memeriksa keanggotaan elemen.

3. *Apa yang dimaksud dengan hashable dalam konteks Python? Sebutkan tiga tipe data yang hashable dan tiga yang tidak hashable. Mengapa kunci dictionary harus hashable?*

- Hashable adalah sifat suatu objek yang memiliki nilai hash tetap selama objek tersebut digunakan dan dapat dibandingkan untuk kesetaraan. Objek hashable dapat digunakan sebagai kunci dictionary.

  Tiga tipe data hashable:

  - `int`
  - `str`
  - `tuple yang seluruh elemennya hashable.`

  Tiga tipe data yang tidak hashable:

  - `list`
  - `dict`
  - `set`

  Kunci dictionary harus hashable agar Python dapat menggunakan nilai hash untuk menemukan dan mengakses pasangan kunci-nilai secara efisien.

4. *Jelaskan mengapa pencarian keanggotaan (operator in) pada set memiliki kompleksitas O(1) sedangkan pada list O(n). Apa implikasi praktisnya untuk data besar?*

   - Pencarian menggunakan operator `in` pada set memiliki kompleksitas rata-rata $O(1)$ karena set menggunakan hash table. Sementara itu, pencarian pada list memiliki kompleksitas $O(n)$ karena Python dapat memeriksa elemen satu per satu sampai menemukan nilai yang dicari.

     Implikasi praktisnya, set lebih efisien untuk memeriksa keanggotaan pada data berukuran besar, sedangkan list membutuhkan waktu lebih lama ketika jumlah elemennya bertambah.

5. *Apa perbedaan antara metode sort() dan fungsi sorted() pada list? Kapan sebaiknya menggunakan masing-masing?*
   - `sort()` merupakan metode list yang mengurutkan elemen secara langsung pada list asli dan tidak menghasilkan list baru sebagai nilai kembalian.
   - `sorted()` merupakan fungsi yang menghasilkan list baru yang sudah terurut tanpa mengubah urutan data asli.
   - `sort()` digunakan ketika data asli boleh diubah, sedangkan `sorted()` digunakan ketika data asli harus dipertahankan.

   Contoh:

```
angka = [3, 1, 2]

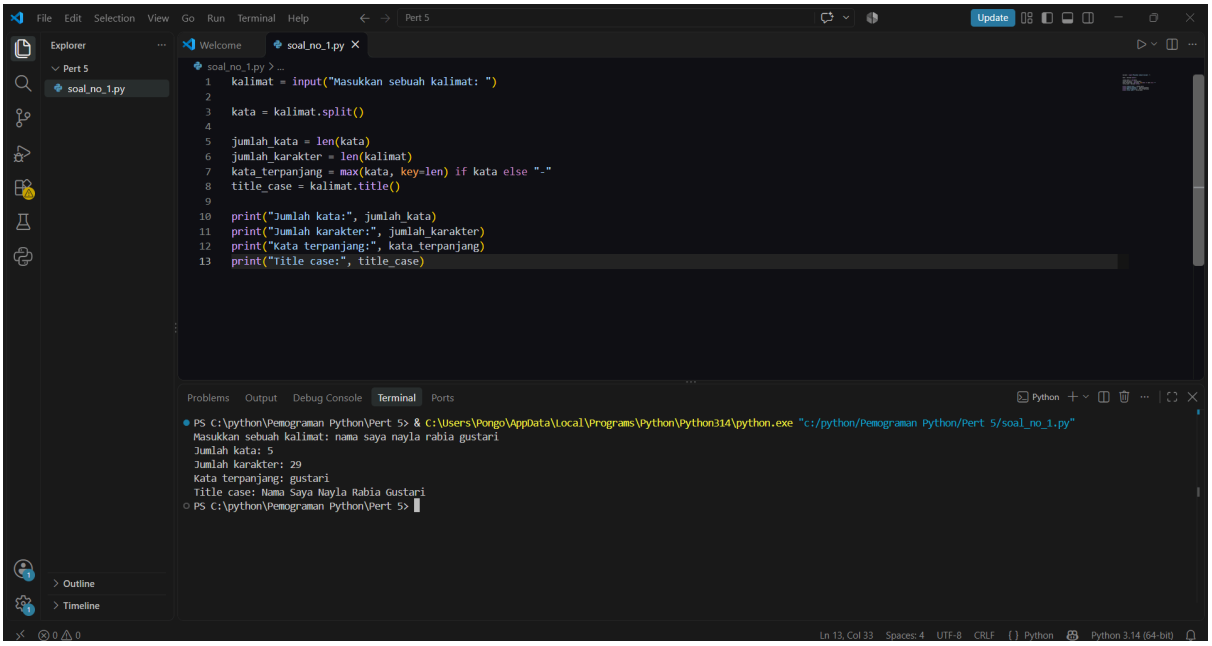
angka.sort()
print(angka)   # [1, 2, 3]

data = [3, 1, 2]
hasil = sorted(data)

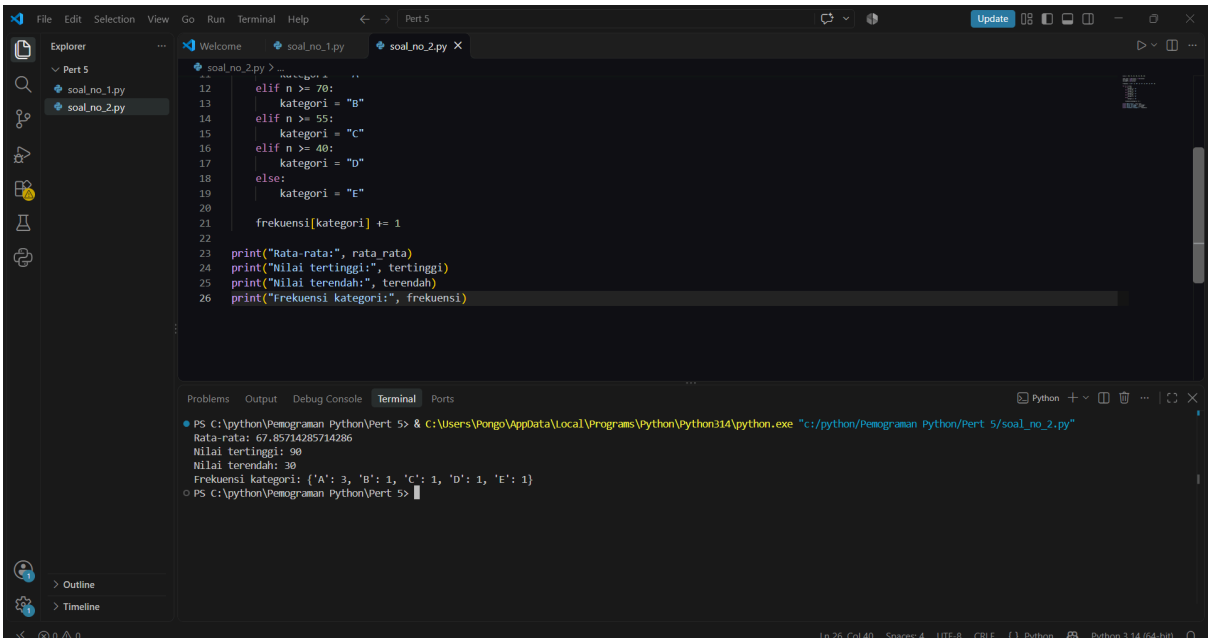
print(data)    # [3, 1, 2]
print(hasil)   # [1, 2, 3]
```

B. SOAL PRAKTIK

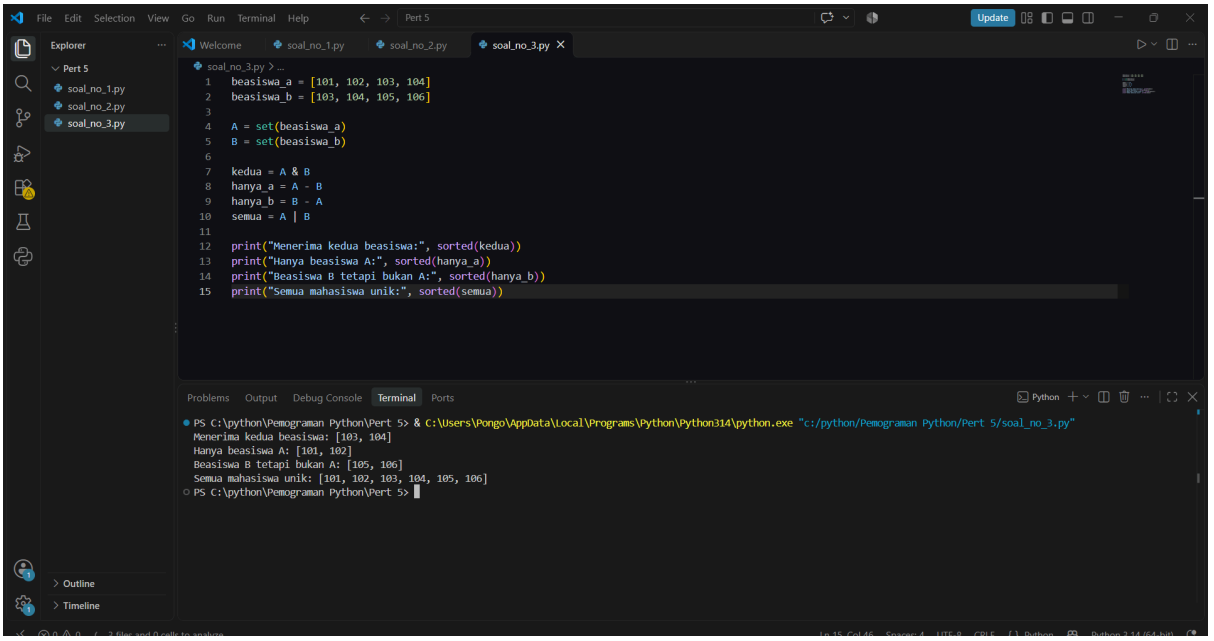
1. *Buatlah program yang menerima sebuah kalimat dari pengguna, lalu: (a) hitung jumlah kata, (b) hitung jumlah karakter, (c) tampilkan kata terpanjang, (d) tampilkan kalimat dengan setiap kata di kapitalisasi (title case).*



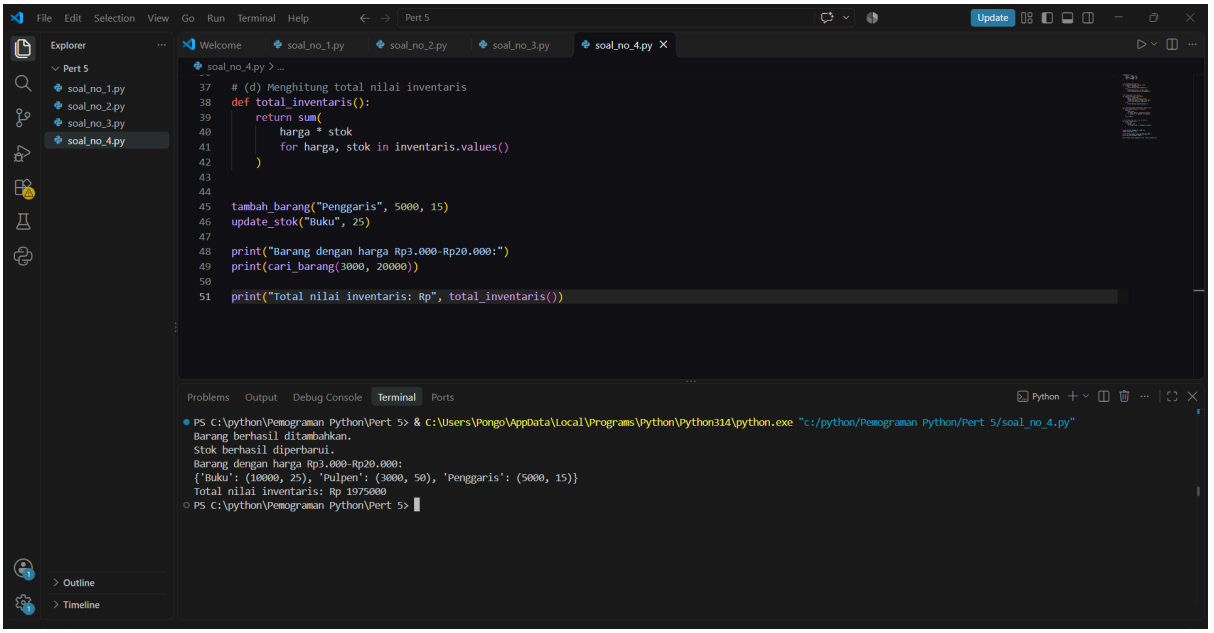
2. *Buatlah program yang menerima list nilai mahasiswa, lalu: (a) hitung rata-rata, (b) tentukan nilai tertinggi dan terendah, (c) kelompokkan nilai ke dalam kategori A/B/C/D/E, (d) tampilkan frekuensi setiap kategori menggunakan dictionary.*



3. *Diberikan dua list yang berisi ID mahasiswa penerima beasiswa A dan beasiswa B, gunakan operasi set untuk: (a) temukan mahasiswa yang menerima kedua beasiswa, (b) temukan yang hanya menerima beasiswa A, (c) temukan yang menerima beasiswa B tapi tidak A, (d) temukan semua mahasiswa unik yang menerima beasiswa.*



4. *Buatlah dictionary yang menyimpan data inventaris toko: nama barang sebagai kunci dan (harga, stok) sebagai nilai tuple. Implementasikan fungsi untuk: (a) menambah barang baru, (b) mengupdate stok, (c) mencari barang dengan harga dalam rentang tertentu, (d) menghitung total nilai inventaris.*



```
# (d) Menghitung total nilai inventaris
def total_inventaris():
    return sum(
        harga * stok
        for harga, stok in inventaris.values()
    )

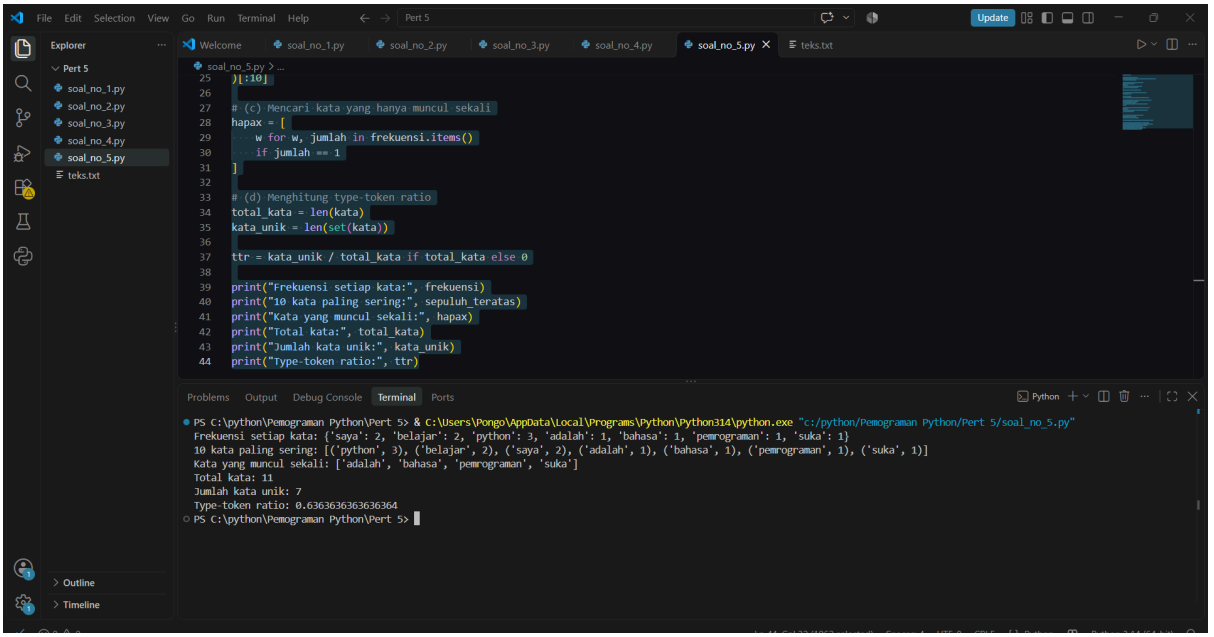

tambah_barang("Penggaris", 5000, 15)
update_stok("Buku", 25)

print("Barang dengan harga Rp3.000-Rp20.000:")
print(cari_barang(3000, 20000))

print("Total nilai inventaris: Rp", total_inventaris())
```

```
PS C:\python\Pemograman Python\Pert 5> & C:\Users\Pongo\AppData\Local\Programs\Python\Python314\python.exe "c:/python/Pemograman Python/Pert 5/soal_no_4.py"
Barang berhasil ditambahkan.
Stok berhasil diperbarui.
Barang dengan harga Rp3.000-Rp20.000:
{'Buku': (10000, 25), 'Pulpen': (3000, 50), 'Penggaris': (5000, 15)}
Total nilai inventaris: Rp 1975000
PS C:\python\Pemograman Python\Pert 5>
```

5. *Buatlah program yang membaca teks dari file, lalu menggunakan kombinasi struktur data (string, list, set, dict) untuk: (a) menghitung frekuensi setiap kata, (b) menampilkan 10 kata paling sering muncul, (c) mengidentifikasi kata yang hanya muncul sekali (hapax legomena), (d) menghitung rasio kata unik terhadap total kata (type-token ratio)*



```
)[:10]

# (c) Mencari kata yang hanya muncul sekali
hapax = [
    w for w, jumlah in frekuensi.items()
    if jumlah == 1
]

# (d) Menghitung type-token ratio
total_kata = len(kata)
kata_unik = len(set(kata))

ttr = kata_unik / total_kata if total_kata else 0

print("Frekuensi setiap kata:", frekuensi)
print("10 kata paling sering:", sepuluh_teratas)
print("Kata yang muncul sekali:", hapax)
print("Total kata:", total_kata)
print("Jumlah kata unik:", kata_unik)
print("Type-token ratio:", ttr)
```

```
PS C:\python\Pemograman Python\Pert 5> & C:\Users\Pongo\AppData\Local\Programs\Python\Python314\python.exe "c:/python/Pemograman Python/Pert 5/soal_no_5.py"
Frekuensi setiap kata: {'saya': 2, 'belajar': 2, 'python': 3, 'adalah': 1, 'bahasa': 1, 'pemrograman': 1, 'suka': 1}
10 kata paling sering: [('python', 3), ('belajar', 2), ('saya', 2), ('adalah', 1), ('bahasa', 1), ('pemrograman', 1), ('suka', 1)]
Kata yang muncul sekali: ['adalah', 'bahasa', 'pemrograman', 'suka']
Total kata: 11
Jumlah kata unik: 7
Type-token ratio: 0.6363636363636364
PS C:\python\Pemograman Python\Pert 5>
```